Intro to Robotics:Wizardry Chess

Ricardo Metral
Computer Science
Intro to Robotics
Orlando, USA
ri828586@ucf.edu

Kevin Rivera
Mechanical Engineering
Intro to Robotics
Orlando, USA
ke695526@ucf.edu

Max Eiken
Mechanical Engineering
Intro to Robotics
Orlando, USA
ma388407@ucf.edu

In this project, we wanted to create a fully autonomous chessboard that can be played using just your voice. We were inspired by Harry Potter and wanted to mimic the idea of Wizardry Chess, as introduced in the first movie of the series. The pieces would also track a form of "loyalty" to a player based on the player's rating during the game. Using this rating, "insubordinate" pieces would purposefully disregard the players' input and make their own decision.

I. Introduction

In the context of robotics research, this project drew a lot of inspiration from existing systems. The Cartesian gantry robot that is hidden under the board is very similar to CNC and 3D printers in application, and has a belt-driven plate that runs across 2 axes driven by stepper motors. Likewise, there are a few projects online that use systems in order to play chess using similar hidden robots, although the most common designs utilize an H-bot driven by the use of one long belt rather than two, as in our system.

In terms of the path planning capabilities of the robot, it draws resemblance to grid-based navigation algorithms commonly utilized in automated warehouse environments and swarm robotics. Because the chessboard is a constrained environment with fixed obstacles (other pieces), the system must solve for the shortest path while avoiding collisions, a task frequently handled by A-Star or Dijkstra's algorithm. In our case, we are using a Breadth-First-Search algorithm, and given that we have a closed world environment, we do not require the use of sensing to determine the position of the magnet or other pieces.

Finally, in regard to the loyalty system, we incorporated a chess engine named Stockfish into our own algorithm to update the loyalty after every move a player makes. Stockfish uses minmax trees with heuristics at every node to evaluate the position. In 2023, Stockfish switched to using a neural network to evaluate the positions, instead of hand-crafted heuristics. The neural network used by Stockfish is called an efficiently updatable neural network. It also uses an alpha-beta pruning technique to search through the minimax tree.

II. Method

The chessboard itself uses a Raspberry Pi 5 as the main point of computation. Alongside the pi, we used two stepper motors, two motor drivers, one servo motor, and a power supply to power all the hardware alongside the gantry. All development of the software is contained in the GitHub repository, which will be linked in the results section. The Raspberry Pi was set up as a bare-bones Ubuntu 24.04 server to conserve the Pi's scarce resources by focusing as much compute as possible on running the project. This helps because rather than running the entire Raspbian OS in the background, all the Pi needs to do is run one terminal session, which we can access via the SSH protocol. Without the bloat that comes with an entire operating system, the Pi was able to handle running multiple processes required by ROS and Docker in a much more efficient manner.

The entire system is a combination of multiple different algorithms used in conjunction with each other, implemented together using ROS2. The ROS2 workspace is separated into seven different packages that all work in conjunction with each other. Those seven packages are combined into five different subsystems, which are: navigation, planning, motor schema, board state, and player input. The mechanics of each subsystem will be described below, and a graphic illustrating the intercommunication between each of the subsystems will be presented at the end of this section.

A. Navigation

The navigation subsystem is under the chess_nav directory in the repository. As briefly mentioned in the introduction, we are using a BFS algorithm to return a set of waypoints for the motor schema to follow. The main issue that comes with writing navigation in this project is obstacle avoidance. With multiple pieces on the board, it is important to consider three main things: not bumping other pieces off their current position, not getting too close to other pieces and accidentally towing them alongside the main piece we are moving, and handling captured pieces.

To remedy the first issue, the navigation algorithm will only search for paths on the edges between all the squares on the board, rather than go through the squares themselves. This approach never has the issue of running into pieces, since a piece will never be sitting on a line between squares; on the other hand, this approach can add more computation than necessary in some cases. If a piece is moving only one square in any direction, it is much easier to simply move the gantry to that square as opposed to finding a very short path on the lines between squares, and then moving exactly along those lines to bring the piece to the center of the square.

The second issue we face is that if we get too close to another piece, the magnetic field of both the piece and the magnet under the board might catch another piece by accident. Part of the solution to this problem is the same as the first issue, where traveling between squares helps us always keep a certain distance from all other pieces. Additionally, this is where the

importance of the servo motor appears. Using the servo, we can change the height of the magnet under the board at any point in time. This allows the magnet to act as “off” and “on” and never drag pieces unintentionally.

The third and final issue we faced was how to handle moves that capture an opponent's pieces. The answer to this issue is solved in two parts: adding a graveyard to our board and using our existing functions to handle the navigation of the captured piece. A normal chessboard is an 8x8 grid, but our board is actually a 10x10 grid to account for the graveyard. The playable area during the game is only in the normal 8x8 grid; however, if a piece is captured, we call the _move_piece() function in navigation.py to our destination square in the graveyard. This will do all the same navigation and movement as previously described before the attacking piece is moved, and move the captured piece into the graveyard. After the captured piece is out of the way, the attacking piece will be moved to the appropriate square, and it will be the opposing player’s turn.

B. *Planning*

This subsystem is what orchestrates every other subsystem in the project. It handles the game state, calling for player input and calling the navigation subsystem. This node is both a client to the player_input_service and a publisher to the player_move topic.

After the game is instantiated and all nodes are set up, the planner first calls the player_input_service, waiting for player input via STT. Once the service returns, the planner rates the move and updates the loyalty. Lastly, it switches the player’s turns and publishes to the player_move topic, which will initiate the navigator to begin searching for a path and move the motors once a path is found. It will repeat this loop until the player_input_service returns a message that signals the end of the game. After the game ends, the planner will send a trigger to reset the board and destroy the node.

C. *Motor Schema*

This subsystem is contained under the *chess_motor_schema* package in the *gantry.py* file. The motors were implemented using the gpiozero library in Python to access the Pi’s GPIO pins. There are two main classes contained in this file, one for the stepper motors and one for the entire gantry itself. A slight difference between this subsystem and the others is that the motor schema does not use ROS2 at all. To avoid the overhead of ROS topics, the gantry class is directly included in the navigation subsystem to make motor commands. This allows for much more control over the motor and allows for emergency stops if necessary.

D. *Board State*

Contained under the chess_board_state package in the file board_state.py, this subsystem uses the Python chess library to access many high-level functions regarding a chess game and handles the underlying board state during the program's lifecycle. This node works by simply subscribing to the player_move topic and calling our update function every time a new message is published to the topic. This package also provides many high-level functions that are called in other subsystems, such as check_move_valid(), which checks to make sure a move is valid and analyze_board(), which is used to get the centipawn score to update the loyalty in the planner subsystem.

E. *Player Input*

The last subsystem used in this project. It is contained under the chess_player_input package in the stt_player_input.py file. As previously mentioned, player input is taken in using a speech-to-text model named Vosk. Vosk has a built-in Python package to make it easy to download the model and use it in any file. This package is only a service via the player_input topic. The service works by using multithreading to listen for player input and wait for service calls.

Using one thread, the node will always listen for the next player to press their respective button and state their move. Using the board_state node, we can check whether the returned move is valid. If not valid, simply wait for speech input again and repeat the process. Once a valid move is returned, append it to a buffer containing all the moves and wait for the next button press again.

In the second thread, we wait for a call from the client to reach the ROS2 service. Once a call has been received, all this node has to do is read from the move buffer if there is a move at the proper index. If not, the thread will sleep until the correct player inputs a move using the first thread. The buffer being updated will alert the second thread to wake and return the move in the buffer to the client.

Although more complex, this approach allows for future work on the project to be much more scalable. For example, if we want to hold the match history so that a player can review the game move by move, holding everything in this buffer would make that much simpler. Additionally, if a player wants to make a pre-move, it would be much easier to allow for that with this approach already implemented.

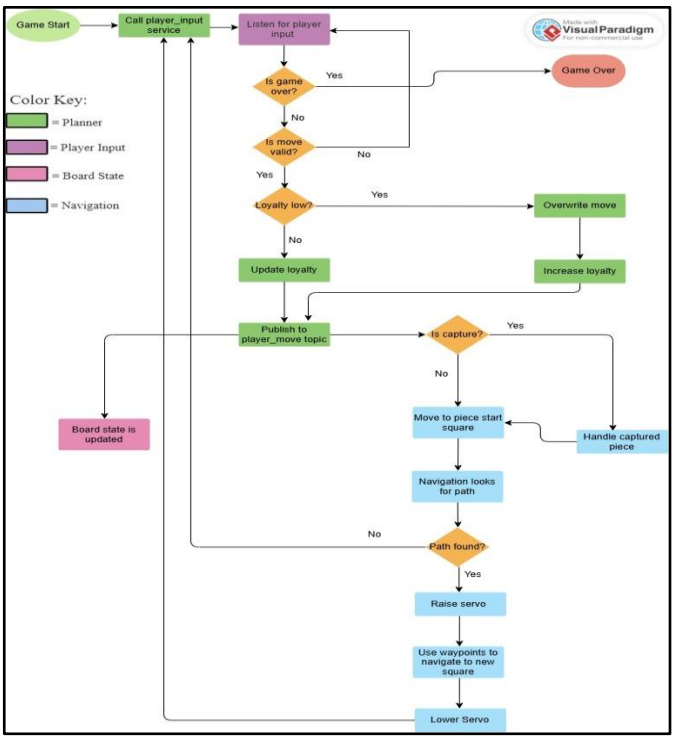

Fig. 1. System Arhitehcture and Control Logic

F. *Mechanical Assembly*

In terms of the mechanical assembly of the robot itself, all parts were purchased, and the relative cost of the robot was not too high. The main parts of the robot assembly were three 2040 aluminum extrusion rods, two stepper motors, a set of belts, pulleys, and rollers. Limit switches served as end stop sensors, which defined the boundaries of the gantry's motion as well as input buttons for triggering the text-to-speech functionality. The final design uses a servo-actuated permanent Neodymium magnet mounted on a lever arm to allow the magnet to engage and disengage with the chess pieces. Components can be seen below.

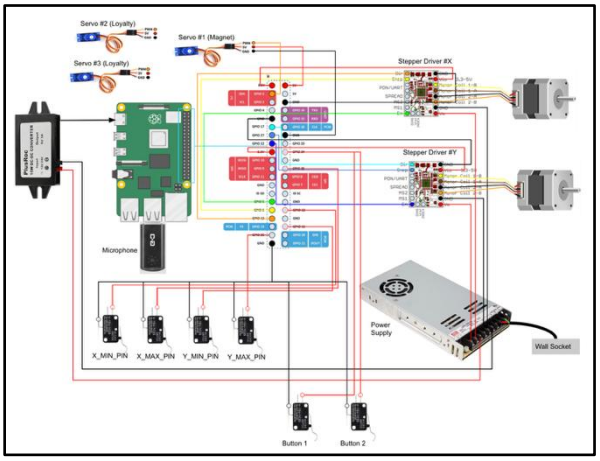

Fig. 2. Electrical Schematic

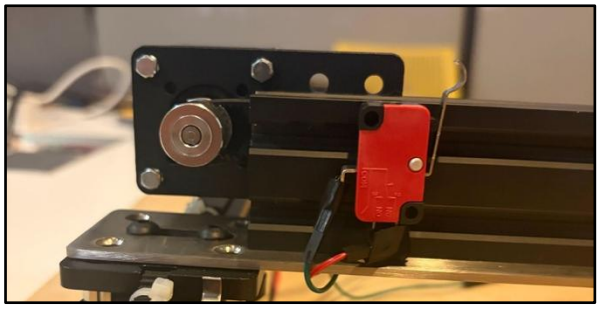

Fig. 3. Motor, Pulley, and Switch Setup

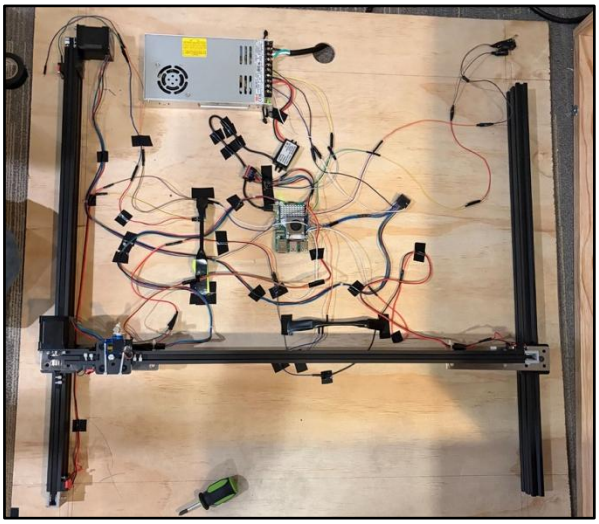

Fig. 4. Finalized Robot Assembly

When designing the housing for the Gantry, both the table and the Gantry role model are modeled in CAD prior to purchasing hardware and creating the physical housing. This was to identify dimensional constraints and just make sure everything fit properly within the housing. The final housing or enclosure was constructed using Ashwood, exhibiting structural integrity while having a visual appeal. The dimensions of the table are 3 ft by 3 ft with a total height of 6 in. With 5 in of internal space to house the Gantry system, which is more than sufficient. The table was designed with a removable top. This was done using threaded inserts and bolts. This allows us to remove the top half of the table from the bottom, making it much easier to access the hardware by simply removing a couple of bolts. The intention of the design was so that it could be as close to plug-and-play as possible.

III. Results

Demo Video:
https://drive.google.com/file/d/1FdIVNJVsC7M01BdKD7TQzL181ZxDv5et/view?usp=drive_link

Github Link:
https://github.com/RickyMetral/Wizardry_Chess_ws

IV. Discussion

The end result of the project was very close to complete, but unfortunately, we were unable to fully finish the project. All the packages and subsystems work except the navigation component. The navigation issue seems to be within the BFS algorithm itself, where the code can never find a valid path to the destination square and thus returns no waypoints for the motors to move. The communication between all the subsystems and their individual functions is fully working as intended, however. Another issue we ran into was wiring all the components together. We had to be very careful with how we wired all the components on the board to avoid wires getting tangled with the gantry's movement. As a result, if some wires in the future move slightly, things might get tangled.

On a brighter note, we were a little worried that the microphone would have issues picking up voice commands since it was enclosed in a box; however, it works even better than expected and is quite good at picking up commands. Another great aspect of the project is that it can be played from anywhere with an outlet and cell service. The Pi is configured to automatically connect to a phone's hotspot, so as long as that phone's hotspot is saved in the Pi, it will automatically connect, and from there, a user can SSH into the Pi and run the launch file containing the project.

In the early design iteration for the project, the team used and purchased an electromagnet for the piece manipulation. When implementing this into the design, we very quickly realized that electromagnets produce excessive heat generation and feared that this may pose a fire risk, given that the electromagnet is enclosed within a wooden structure. Given this safety concern, the team switched to a small servo with a lever arm and a magnet attached to it.

Lastly, during the development of the robot, speech-to-text was not always enabled, so the project has a second 'mode' using a live Lichess game instead of STT. This allows the user to simulate a Lichess game in physical reality using the board automatically. Unfortunately, the loyalty system does not work in this implementation of the board, since moves are made directly in the Lichess server; they cannot be overwritten using the loyalty system, thus this configuration should be viewed more as a bonus than a main feature.

V. Conclusion

Other than fixing our navigation algorithm, the project does have some features that we would like to add. One of the big downsides that the project currently has is that it is not easy to run the project without instructions and a little technical knowledge. Requiring the user to SSH into the Pi, run the Docker container, and run the launch file is not seamless and could be improved using a cron job. With some improvements in the code, instead of ending the script when a game is finished, we could simply reset the board state to allow repetitions of games. Then, by running all the necessary code on startup with a cron job, the project would simply be plug-and-play, no technical knowledge necessary.

Some other improvements include adding more modes, such as PvE, where the player plays against an AI, or even an even EvE, allowing two bots to play against each other, so it looks like the gantry is magically playing itself. Additionally, we could make a PCB to aid in keeping our wires more organized and avoid them getting tangled.

VI. Contribution

In terms of contribution I believe that the work was split up relatively evenly between the three of us. We separated the project into three main components, the software, the gantry construction and the table construction. Max took charge of doing all the research, buying all the parts for the gantry, and figuring out the electrical wiring. Kevin took charge of making the actual physical box with woodworking and taking dimensions of everything. I, Ricardo, took charge of writing all the software for the gantry and game. Our combined knowledge of our individual fields really helped us build a great project!

VII. References

[1] [1] J. Zhang, "Chess Movement," YouTube, Apr. 25, 2018. [Online]. Available: https://www.youtube.com/watch?v=OehUH4OccsU.

[2] [2] Greg06, "Automated Chessboard," Instructables. [Online]. Available: https://www.instructables.com/Automated-Chessboard/.

[3] [3] J. Alshanti and M. Alshanti, "Wizard's Chess," Readymag. [Online]. Available: https://readymag.website/u2481798807/5057562/.